

PRÁCTICA 2

Implementación del Protocolo I2C en ESP32-C6

Interfaces de HW y SW

MicroPython · I2C · LCD

1. Objetivo

El objetivo de esta práctica es implementar y validar el protocolo de comunicación I2C en un microcontrolador ESP32-C6 programado con MicroPython. De manera complementaria, se integran los siguientes conceptos:

- Uso de interrupciones externas (IRQ) para la captura de eventos de botones físicos.
- Configuración de resistencias internas PULL-UP y PULL-DOWN por software.
- Implementación de un sistema de gestión de energía mediante el modo LIGHTSLEEP.
- Uso de Timers para conteo de pulsaciones múltiples (multigestos).
- Visualización de información en una pantalla LCD 16x2 controlada vía I2C.
- Aplicación de un mecanismo de antirrebote (debounce) por software.

1.1 Objetivos Específicos

- Configurar el bus I2C del ESP32-C6 a 400 kHz y verificar la detección del módulo PCF8574 en la dirección 0x27.
- Implementar interrupciones externas en dos GPIO con configuraciones PULL-UP y PULL-DOWN para capturar eventos de botones sin polling.
- Desarrollar un sistema de multigestos basado en Timer que distinga 1, 2 o 3+ pulsaciones en una ventana de 1 segundo para controlar la animación de la LCD.
- Aplicar el modo LIGHTSLEEP del ESP32-C6 para reducir el consumo energético, con capacidad de despertar mediante interrupción de GPIO.
- Implementar un algoritmo de antirrebote por software usando marcas de tiempo (ticks_ms) para evitar lecturas falsas en los botones mecánicos.

1.2 Alcance de la Práctica

El sistema resultante demuestra la integración de periféricos de entrada/salida, manejo de energía y comunicación serial sincrónica en un entorno de prototipado embebido. El alcance de esta práctica comprende exclusivamente la implementación en firmware con MicroPython sobre el ESP32-C6, la validación funcional del protocolo I2C, el control de la pantalla LCD 16x2 y la gestión de eventos mediante interrupciones y timers por software. No se incluye diseño de PCB, comunicación inalámbrica ni almacenamiento persistente de datos.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
------------	----------	-------------

ESP32-C6	1	Microcontrolador principal con soporte I2C, WiFi y BLE
Pantalla LCD 16x2 con módulo I2C (PCF8574)	1	Salida visual del sistema
Push Button	2	Entradas de usuario (BTN1 y BTN2)
Resistencias 4.7 kΩ	2	Pull-up externas para líneas I2C (SDA/SCL)

2.2 Mapa de Pines

Variable	GPIO ESP32-C6	Resistencia Interna	Función
btn1	GPIO 1	PULL_UP	Modo LIGHTSLEEP
btn2	GPIO 4	PULL_DOWN	Control multigestos LCD
SCL (I2C)	GPIO 7	4.7 kΩ externas	Señal de reloj I2C
SDA (I2C)	GPIO 6	4.7 kΩ externas	Datos I2C

2.3 Esquema de Conexión

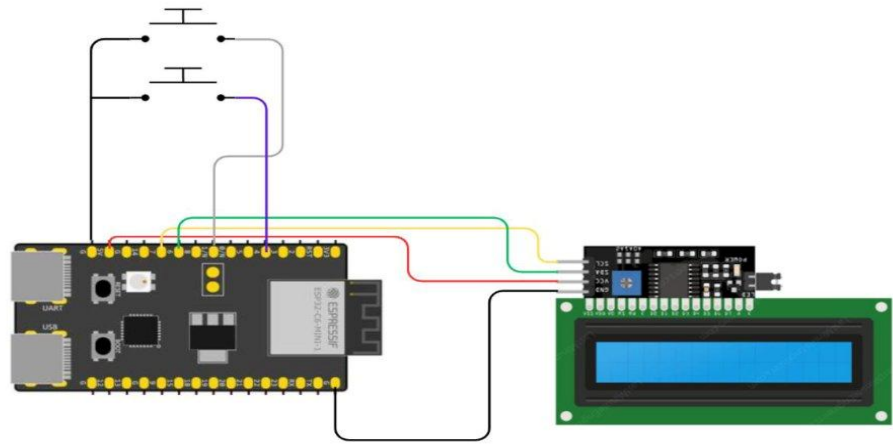


Figura 1. Esquema general del sistema (ESP32-C6 + LCD I2C + Push Buttons)

2.4 Pruebas Eléctricas Realizadas

Antes de cargar el firmware se realizaron las siguientes verificaciones eléctricas para garantizar la integridad del circuito y prevenir daños al microcontrolador:

- **Niveles de voltaje en pines GPIO:** El ESP32-C6 opera a 3.3 V en todos sus pines de I/O. Se verificó con multímetro que la alimentación de la placa fuera de 3.3 V y que

ninguna señal externa superara ese umbral, ya que voltajes mayores (p. ej. 5 V) dañan permanentemente los pines del microcontrolador.

- **Alimentación de la LCD:** La pantalla LCD y su módulo I2C (PCF8574) se alimentaron a 5 V (VIN de la placa), mientras que las líneas SDA y SCL se mantuvieron a 3.3 V gracias a las resistencias de pull-up de 4.7 kΩ conectadas a 3.3 V. Se confirmó que el módulo PCF8574 es tolerante a 3.3 V en sus líneas de datos.
- **Resistencias de pull-up I2C:** Se midió la resistencia entre SDA/SCL y VCC (3.3 V) obteniendo ~4.7 kΩ en ambas líneas, confirmando la correcta instalación. Valores muy bajos (menos de 1 kΩ) causarían consumo excesivo; valores muy altos (más de 10 kΩ) degradan la señal a 400 kHz.
- **Configuración PULL-UP/PULL-DOWN de botones:** Con los botones en reposo se midió: GPIO1 en ~3.3 V (PULL-UP activo) y GPIO4 en ~0 V (PULL-DOWN activo). Al presionar cada botón los niveles se invirtieron correctamente, confirmando el funcionamiento antes de activar las interrupciones.

Todas las mediciones fueron realizadas con multímetro digital antes de energizar el sistema completo, siguiendo buenas prácticas de prototipado para sistemas embebidos.

3. Arquitectura del Software

El proyecto está organizado en tres archivos Python que separan responsabilidades de manera clara:

Archivo	Responsabilidad
p2.py	Lógica principal: GPIO, interrupciones, timer, bucle de visualización y modo sleep.
i2c_lcd.py	Driver de bajo nivel para la LCD: inicialización del controlador HD44780 vía I2C y escritura de bytes nibble por nibble.
lcd_api.py	Capa de abstracción (API) de la LCD: comandos estándar (clear, home, move_to, putstr) independientes del hardware.

4. Descripción Técnica Detallada

4.1 Capa de Abstracción LCD — lcd_api.py

Este archivo define la clase base LcdApi, que encapsula el conjunto de comandos del controlador HD44780 (el más común en pantallas LCD de caracteres). No contiene lógica de hardware; su objetivo es definir una interfaz limpia que los drivers concretos puedan implementar.

Constantes del controlador HD44780

Cada constante representa un comando o máscara de bits que se envía directamente al controlador de la LCD:

```

LCD_CLR      = 0x01    # Limpiar toda la pantalla y regresar cursor a inicio
LCD_HOME     = 0x02    # Regresar cursor al origen sin borrar contenido
LCD_ENTRY_MODE = 0x04  # Modo de entrada (dirección del cursor)
LCD_ENTRY_INC = 0x02    # Auto-incremento: cursor avanza a la derecha
LCD_ON_CTRL   = 0x08    # Control de encendido de display
LCD_ON_DISPLAY = 0x04  # Bit: encender el display
LCD_DDRAM    = 0x80    # Dirección base de la RAM de datos del display

```

Método `set_cursor / move_to`

Posiciona el cursor en una columna y fila específicas. La segunda fila comienza en la dirección 0x40 de la memoria DDRAM, por lo que se suma ese offset cuando row=1:

```

def set_cursor(self, col, row):
    addr = col & 0x3F          # Máximo 63 columnas
    if row & 1:                # Si la fila es impar (fila 1)
        addr += 0x40          # Saltar a la segunda línea en DDRAM
    self.hal_write_command(self.LCD_DDRAM | addr)

```

Método `putstr`

Escribe una cadena de texto carácter por carácter, convirtiendo cada uno a su código ASCII antes de enviarlo a la LCD:

```

def putstr(self, string):
    for char in string:
        self.hal_write_data(ord(char)) # ord() convierte 'A' -> 65

```

4.2 Driver I2C LCD — `i2c_lcd.py`

Este archivo implementa concretamente la comunicación entre el ESP32-C6 y la pantalla LCD a través del módulo adaptador I2C (PCF8574). La pantalla LCD utiliza un bus de datos de 4 bits, por lo que cada byte se envía en dos partes (nibbles).

Constantes de comunicación

```

LCD_I2C_ADDR = 0x27    # Dirección I2C del módulo PCF8574 (jumpers A0-A2 = 0)
LCD_CHR      = 1       # Modo datos: enviar un carácter a mostrar
LCD_CMD      = 0       # Modo comando: enviar instrucción al controlador
LCD_BACKLIGHT= 0x08    # Bit de control del backlight (retroiluminación)
ENABLE       = 0b00000100 # Bit EN: pulso que latch-ea datos en la LCD

```

Inicialización de la LCD

El proceso de inicialización sigue la secuencia especificada por el datasheet del HD44780 para el modo de 4 bits. Se envían tres veces el nibble 0x03 (modo 8-bit simulado) antes de cambiar a modo 4-bit real (0x02):

```
def __init__(self, i2c, i2c_addr, num_lines, num_columns):
    time.sleep_ms(20)          # Esperar estabilización de la LCD
    self.hal_write_init_nibble(0x03) # Primer intento de reset
    time.sleep_ms(5)
    self.hal_write_init_nibble(0x03) # Segundo intento
    time.sleep_ms(1)
    self.hal_write_init_nibble(0x03) # Tercer intento
    self.hal_write_init_nibble(0x02) # Cambiar a modo 4 bits
    # Configurar: 2 líneas, fuente 5x8
    self.hal_write_command(self.LCD_FUNCTION | self.LCD_FUNCTION_2LINES)
    self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY)
    self.hal_write_command(self.LCD_CLR) # Limpiar pantalla
    time.sleep_ms(2)
    self.hal_write_command(self.LCD_ENTRY_MODE | self.LCD_ENTRY_INC)
```

Escritura de byte en modo 4 bits

Cada byte (8 bits) se divide en dos nibbles de 4 bits. Primero se envía el nibble alto (bits 7-4) y luego el nibble bajo (bits 3-0). Para cada nibble se genera un pulso en el pin ENABLE que indica a la LCD que debe leer los datos:

```
def hal_write_byte(self, data, mode):
    high = mode | (data & 0xF0) | self.backlight # Nibble alto
    low  = mode | ((data << 4) & 0xF0) | self.backlight # Nibble bajo
    # Enviar nibble alto con pulso ENABLE
    self.i2c.writeto(self.i2c_addr, bytes([high | self.ENABLE]))
    self.i2c.writeto(self.i2c_addr, bytes([high])) # Bajar ENABLE
    # Enviar nibble bajo con pulso ENABLE
    self.i2c.writeto(self.i2c_addr, bytes([low | self.ENABLE]))
    self.i2c.writeto(self.i2c_addr, bytes([low])) # Bajar ENABLE
```

4.3 Lógica Principal — p2.py

Este es el archivo central del proyecto. Integra todos los subsistemas: GPIO, I2C, interrupciones, timers, modo de ahorro de energía y la animación de texto en la LCD.

4.3.1 Variables Globales de Estado

El sistema mantiene su estado a través de un conjunto de variables globales que son modificadas por las interrupciones y leídas por el bucle principal:

```
msj      = "SALUDOS"      # Mensaje a mostrar en la LCD
lcd_width = 16            # Ancho de la pantalla en caracteres
derecha  = False          # False = barrido de derecha→izquierda (por defecto)
hibernar  = False         # True cuando se debe entrar en modo sleep
barrido   = True          # True = el texto se desplaza, False = texto estático
contador  = 0             # Cuenta los pulsos del BTN2 en la ventana de tiempo
```

El mensaje se prepara con espacios en blanco a cada lado para lograr el efecto de entrada y salida suave en la pantalla:

```
msj_padded = " " * lcd_width + msj + " " * lcd_width
# Resultado: '          SALUDOS          '
# Esto permite que el texto 'entre' desde el borde y 'salga' por el otro
```

4.3.2 Configuración de GPIO e Interrupciones

La función configIO() inicializa los pines de los botones y les asigna handlers de interrupción. Se utiliza un diccionario (btns) para centralizar la configuración de cada botón, lo que facilita agregar más entradas en el futuro:

```
btns = {
    "b1": {"pin": 1, "pull": Pin.PULL_UP,
           "trigger": Pin.IRQ_FALLING, "handler": def_sleep},
    "b2": {"pin": 4, "pull": Pin.PULL_DOWN,
           "trigger": Pin.IRQ_FALLING, "handler": def_mov_lcd}
}
```

Notas importantes sobre la configuración:

- BTN1 usa PULL_UP: el pin está en HIGH en reposo; al presionar el botón conecta a GND y cae a LOW (flanco de bajada = IRQ_FALLING).
- BTN2 usa PULL_DOWN: el pin está en LOW en reposo; al presionar sube a HIGH. Sin embargo, el trigger también es IRQ_FALLING porque el botón genera un flanco de subida seguido de uno de bajada y se detecta en la bajada.
- IRQ_FALLING detecta el momento exacto en que el botón se presiona y el voltaje cae.

4.3.3 Antirrebote (Debounce) por Software

Los botones mecánicos generan múltiples señales eléctricas en milisegundos al presionarse (efecto rebote). Sin antirrebote, una sola pulsación puede registrarse como 5-20 eventos. La solución implementada usa marcas de tiempo:

```
last_ms = {} # Diccionario: guarda el último timestamp por nombre de botón

def antirrebote(nombre, intervalo_ms=200):
    now = time.ticks_ms() # Tiempo actual en milisegundos
    last = last_ms.get(nombre, 0) # Último tiempo registrado (0 si es nuevo)
    if time.ticks_diff(now, last) < intervalo_ms:
        return False # Muy pronto, ignorar este evento
    last_ms[nombre] = now # Actualizar timestamp
    return True # Evento válido, procesarlo
```

`time.ticks_diff()` se usa en lugar de la resta directa porque los ticks pueden desbordarse (overflow) después de cierto tiempo. Esta función maneja el desbordamiento correctamente.

4.3.4 Sistema de Gestos con Timer

El BTN2 implementa un sistema de multigestos: el comportamiento de la LCD depende de cuántas veces se presiona el botón dentro de una ventana de 1 segundo. Esto se logra con un Timer periódico:

```
t1 = Timer(1) # Timer hardware número 1

def activarTimer():
    global estadot1
    estadot1 = True
    t1.init(mode=Timer.PERIODIC, period=1000, callback=rstConteo)
    # Dispara rstConteo() cada 1000ms (1 segundo)

def rstConteo(t1):
    global contador, derecha, barrido
    miLcd.clear()
    barrido = True
    if contador == 1:
        derecha = True # 1 pulso → izquierda a derecha
    elif contador == 2:
        derecha = False # 2 pulsos → derecha a izquierda
    elif contador >= 3:
        barrido = False # 3+ pulsos → texto estático
    contador = 0 # Reiniciar conteo
    desactivarTimer() # Apagar timer hasta el próximo gesto
```

El handler del BTN2 incrementa el contador y activa el timer si no está activo:


```
def def_mov_lcd(pin):
    global derecha, barrido, contador
    if antirrebote("b1"):
        if not estadot1:
            activarTimer()    # Iniciar ventana de tiempo
            contador += 1      # Contar este pulso
```

Pulsaciones (1 seg)	Variable derecha	Efecto en LCD
1	True	Texto se desplaza de izquierda a derecha
2	False	Texto se desplaza de derecha a izquierda (por defecto)
3 o más	—	Texto estático (barrido = False)

4.3.5 Configuración I2C y LCD

La función `config_i2c()` inicializa el bus I2C del ESP32-C6 y escanea los dispositivos conectados para verificar la conexión. Si el módulo PCF8574 está correctamente conectado, aparecerá en la dirección 0x27:

```
def config_i2c():
    i2c = I2C(scl=7, sda=6, freq=400000)    # 400 kHz = modo Fast I2C
    devices = i2c.scan()                    # Buscar esclavos en el bus
    if devices:
        for d in devices:
            print("I2C device en:", hex(d)) # Mostrar dirección hex
    pines['lcd'] = I2cLcd(i2c, 0x27, 2, 16)
    #             ^      ^ ^
    #             dirección  filas  columnas
```

La frecuencia de 400 kHz (modo Fast) permite transferencias rápidas para una animación fluida sin bloquear el procesador.

4.3.6 Modo LIGHTSLEEP y BTN1

El ESP32-C6 puede reducir su consumo energético suspendiendo el CPU y la mayoría de los periféricos, manteniendo solo la capacidad de despertar ante ciertos eventos. En este proyecto, el modo LIGHTSLEEP se activa con BTN1:

```
def def_sleep():
    global hibernar
    hibernar = True    # Señal para el bucle principal
```

```
# En el bucle principal:
if hibernar:
    miLcd.clear()
```

```

miLcd.move_to(0, 0)
miLcd.putstr("buenas noches")    # Mensaje de despedida
sleep_ms(50)                     # Tiempo para que la LCD muestre el
mensaje
lightsleep()                     # CPU entra en modo bajo consumo
# Aquí el programa se pausa hasta que el mismo BTN1 genere un IRQ
# Al despertar, continúa desde esta línea...
hibernar = False                 # (necesita reset manual en esta versión)

```

Nota: En la implementación actual, `lightsleep()` mantiene activas las interrupciones de GPIO. Al presionar nuevamente BTN1 se genera el IRQ que despierta al ESP32 y el programa continúa.

4.3.7 Bucle Principal de Animación

El bucle `while True` gestiona la animación del texto en la LCD. El desplazamiento se logra moviendo una ventana de 16 caracteres sobre el string `padded`:

```

i = 0    # Índice de inicio de la ventana de visualización
while True:
    if hibernar:
        # ... gestión de sleep (ver 4.3.6)
    else:
        if barrido:
            maximo = len(msj_padded) - lcd_width    # Límite del barrido
            if derecha:
                i = (i + 1) % maximo    # Avanzar índice (izq → der)
            else:
                i = (i - 1) % maximo    # Retroceder índice (der → izq)
            display_text = msj_padded[i : i + lcd_width]
            miLcd.move_to(0, 0)
            miLcd.putstr(display_text)
            sleep_ms(200)    # 5 fps de animación
        else:
            # Texto estático: mostrar posición actual sin mover índice
            miLcd.move_to(0, 0)
            miLcd.putstr(msj_padded[i : i + lcd_width])
            sleep_ms(200)

```

5. Flujo General del Sistema

El siguiente diagrama describe el flujo de ejecución completo del firmware desde el arranque:

```

INICIO → Importar módulos → configIO() → config_i2c()
      ↓
  BUCLE PRINCIPAL

```

```

┌ [hibernar == True?]
└ ┌ mostrar "buenas noches" → lightsleep() → despertar → continuar
    └ [hibernar == False]
        ┌ [barrido == True] → calcular índice i → mostrar ventana → sleep 200ms
        └ [barrido == False] → mostrar texto fijo → sleep 200ms

```

INTERRUPCIONES (paralelas al bucle):

```

BTN1 (GPIO1 ↓) → def_sleep() → hibernar = True
BTN2 (GPIO4 ↓) → antirrebote() → activarTimer() → contador++
Timer(1s)      → rstConteo() → actualizar derecha/barrido → contador=0

```

6. Protocolo I2C — Conceptos Clave

I2C (Inter-Integrated Circuit) es un protocolo de comunicación serie síncrono desarrollado por Philips. Permite conectar múltiples dispositivos con solo 2 cables:

- SDA (Serial Data): línea bidireccional por donde se transmiten los datos.
- SCL (Serial Clock): señal de reloj generada por el maestro (ESP32-C6) para sincronizar la transmisión.

En esta práctica:

- El ESP32-C6 actúa como maestro (master): genera el reloj y dirige la comunicación.
- El módulo PCF8574 de la LCD actúa como esclavo (slave): responde en la dirección 0x27.
- Las resistencias de 4.7 kΩ mantienen las líneas en HIGH cuando no se está transmitiendo (bus I2C open-drain).

Cada transferencia I2C sigue la secuencia: START → [Dirección 7 bits + R/W] → ACK → [Datos] → STOP. En este proyecto todas las transferencias son escrituras (W) del ESP32 hacia la LCD.

7. Dependencias y Bibliotecas

Módulo/Biblioteca	Origen	Uso en el proyecto
machine.Pin	MicroPython stdlib	GPIO, IRQ, PULL_UP/DOWN
machine.I2C	MicroPython stdlib	Bus I2C maestro
machine.Timer	MicroPython stdlib	Ventana de tiempo para gestos
machine.lightsleep	MicroPython stdlib	Modo bajo consumo

time	MicroPython stdlib	ticks_ms() y ticks_diff() para antirrebote
lcd_api.py	Proyecto (theoryCIRCUIT)	Abstracción de comandos LCD HD44780
i2c_lcd.py	Proyecto (theoryCIRCUIT)	Driver físico LCD sobre I2C

8. Conclusiones

Esta práctica integra satisfactoriamente varios conceptos fundamentales de los sistemas embebidos en un único proyecto funcional:

- El protocolo I2C permite conectar la pantalla LCD con solo 2 pines de datos, liberando GPIO para otras funciones y simplificando el cableado.
- El uso de interrupciones externas (IRQ) garantiza que las pulsaciones de los botones sean detectadas inmediatamente, sin importar en qué parte del bucle principal se encuentre el programa.
- El antirrebote por software con marcas de tiempo es una solución eficiente que no requiere retardos bloqueantes (sleep), manteniendo la responsividad del sistema.
- El sistema de multigestos basado en Timer demuestra cómo se pueden crear interfaces de usuario ricas con hardware mínimo (un solo botón con tres comportamientos).
- El modo LIGHTSLEEP es una herramienta poderosa para aplicaciones de bajo consumo, permitiendo que el sistema se despierte ante un evento externo sin consumir energía en el proceso de espera.
- La separación en capas (lcd_api → i2c_lcd → p2) sigue el principio de responsabilidad única y facilita el mantenimiento y la reutilización del código.